



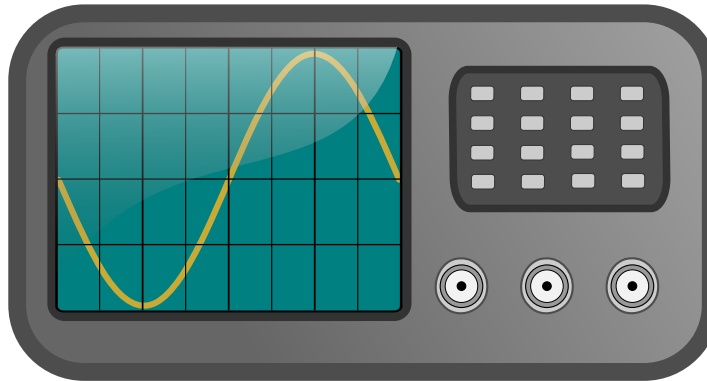
Measuring





Time measurement equipment: hardware

- ▶ The best equipment is an oscilloscope, if you can afford one
- ▶ Allows to time the “Power on” event (connected to a power rail), or any event (connected to a GPIO pin, for example), all this in a very accurate way.
- ▶ Easy to write to a GPIO at all the stages of system booting (we will explain how to do it)
- ▶ Some oscilloscopes are getting affordable. Example: Bitscope Pocket Analyzer (245 AUD, supported on Linux, <https://www.bitscope.com/product/BS10/>)





Measuring with hardware: using an Arduino



<https://arduino.cc>

- ▶ If you don't have an oscilloscope, an Arduino (or any general purpose MCU or MPU board) is a good solution too.
- ▶ The main strength of Arduino is its great ease of use and programming, plus all the hardware support libraries that are available.
- ▶ You can easily connect board pins to the Arduino analog pins, and watch their voltage.
- ▶ Arduino boards are Open Source Hardware. This project is definitely worth supporting!

Arduino Nano

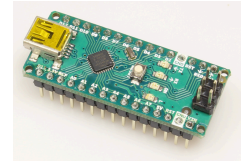
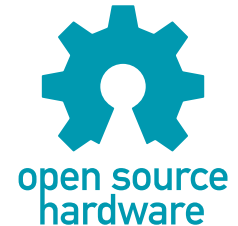


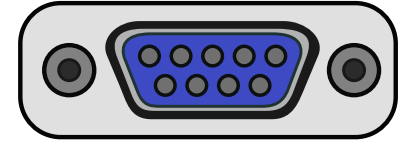
Image credits: https://commons.wikimedia.org/wiki/File:Arduino_nano_isometr.jpg





Time measurement equipment: serial port

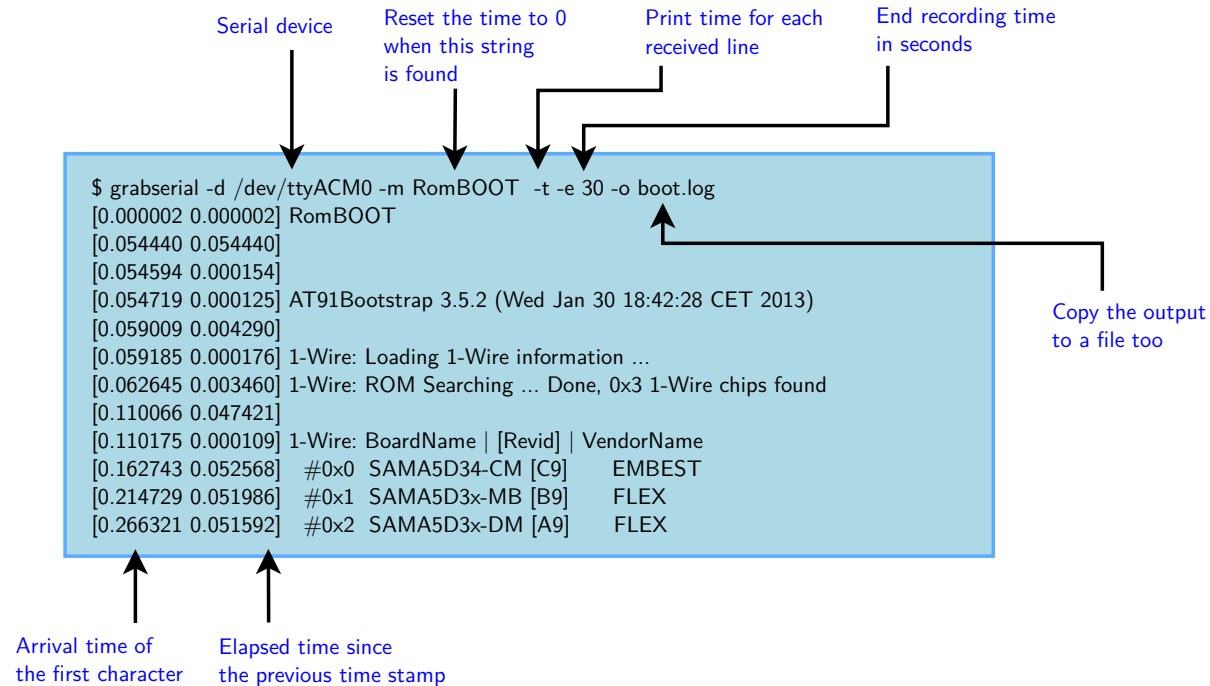
- ▶ Useful when you don't have monitoring hardware, or don't want to make take any risk connecting wires to the hardware.
- ▶ Usually relies on software which times messages received from the board's serial port (serial port absolutely required). Such software runs on a PC connected to the serial port.
- ▶ On the board, requires a real serial port (directly connected to the CPU), immediately usable from the earliest parts of the boot process. Attaching a USB-to-serial dongle to a USB **host** port on the device won't do: USB is available much later and messages go through more complex software stacks (loss of time accuracy).
- ▶ Limitation: won't be able to time the "Power on" event in an accurate way. But acceptable as you can assume that the time to run the ROM code is constant.



- ▶ A Python script to add timestamps to messages received on a serial console.
- ▶ Key advantage: starts counting very early (ROM code — if not silent, bootstrap and bootloader)
- ▶ Another advantage: no overhead on the target, because run on the host machine.
- ▶ Drawbacks: may not be precise enough. Can't measure power up time.
- ▶ Ubuntu package: `grabserial` Otherwise available on <https://github.com/tbird20d/grabserial/>



Using grabserial



Caution: `grabserial` shows the arrival time of the **first character** of a line. This doesn't mean that the entire line was received at that time.



- ▶ You can interrupt `grabserial` manually (with `[Ctrl][c]`) when you have gone far enough.
- ▶ The `-m` (`m`atch start pattern) and `-q` (`q`uit pattern) options actually expect a regular expression. A normal string may not match in the middle of a line.
- ▶ Example: you may have to replace `-m "Starting kernel"` by `-m ".*Starting kernel.*"`.
- ▶ You can store a copy of the output to a file using the `-o` option. No need to copy / paste or redirect the output to keep it.



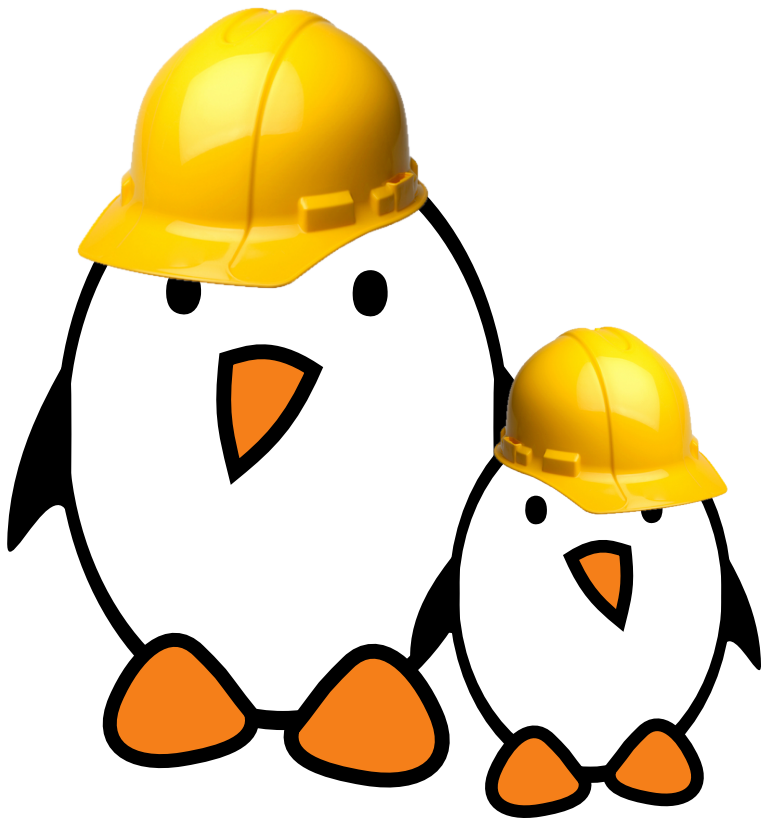
Dedicated measuring resources

Later, we will see specific resources for measuring time

- ▶ `time` for measuring application time
- ▶ `strace` for application tracing
- ▶ `bootchartd` for measuring and tracing the execution of system services.
- ▶ More specifically, `systemd-analyze` if your system is started with *Systemd*.
- ▶ `CONFIG_PRINTK_TIME` and `initcall_debug` for tracing and timestamping kernel code and functions.



Practical lab - Measuring time



Measuring time with software

- ▶ Setting up `grabserial`
- ▶ Modify the video player to log a notification after the first frame is processed.
- ▶ Time the various components of boot time through messages written to the serial console.